Part C: Running this for real on GCP / BigQuery

1. Incremental loading, partitioning and cost. Each extract lands in GCS, then in an append-only `raw_bookings` table: all STRING columns, partitioned by `_batch_date`. Validated rows overwrite their batch's partition in `stg_bookings`, and failed rows go to `rejects`. A single `MERGE` into `fct_bookings` (SQL A3) then updates only `WHEN MATCHED AND s.updated_at > t.updated_at`. That makes re-runs, re-sends and stale out-of-order rows no-ops.

`fct_bookings` is partitioned by `DATE_TRUNC(check_in_date, MONTH)` and clustered by `property_id, booking_channel, booking_status`, which match the monthly, per-property and per-channel reports. At tens of millions of rows, the ways to keep cost down are:

- prune the MERGE target to the batch's check-in date range;
- set `require_partition_filter` on raw and staging tables;
- select only the columns you need;
- give dashboards pre-aggregated monthly tables or materialized views;
- monitor `INFORMATION_SCHEMA.JOBS` and set byte quotas.

2. Orchestration and monitoring. Cloud Composer (Airflow) runs the pipeline, triggered when a file lands in GCS, with a daily schedule as fallback. The steps are:

```
land → validate → stage → DQ gate → MERGE → post-load assertions → refresh aggregates
```

Every step is idempotent, so retries are safe. The DQ gate runs the A6 assertions and the pipeline's `dq_summary`. If the reject rate is too high, or there are duplicate current IDs, missing FX rates or orphan properties, it **stops the run before the MERGE**. Warnings are sent to Slack.

The merge audit (rows inserted, updated, unchanged and stale), freshness (`max(updated_at)`) and volume anomalies go to Cloud Monitoring, with alerts. A missing batch also raises an alert. Rejects stay queryable so the source owner can fix them and re-send.

3. Maintainability.

- **Version control:** Git with PR review.
- **CI:** lint, then `pytest` (including the SQL-vs-Python parity test), then a BigQuery `--dry_run`, then deploy to a dev dataset and run the assertions there, then promote to prod.
- **Infrastructure:** Terraform for datasets, tables and IAM. Service accounts rather than keys.
- **At larger scale:** move the SQL to dbt, so the models, tests, docs and lineage all come from the same code.
- **Runbooks:** backfill, reprocessing rejects, adding a currency, and what to do when the DQ gate fails.
- **Lineage:** each row carries `_batch_id`, so it can be traced back to its source file.

4. Assumptions and trade-offs.

- **Versions:** `updated_at` decides which version is newest. Ties are broken by batch, then file row, and are flagged (1 case: B1021).
- **FX:** the rate is the one effective on the `created_at` date. A bare `$` means USD.
- **Dates:** slash dates are day-first.
- **Rejected rows:** the last valid version stays current. The trade-off is that a bad newer row can't overwrite good data, but a real change it carries waits until the row is fixed and re-sent.
- **Warnings, not rejects:**
  - `created_at` later than the batch file date. The file date looks like a label rather than a cut-off. Configurable.
  - Revenue that doesn't equal rate × nights, because amendments restate it.
  - The property under renovation.
- **Reporting:** revenue means confirmed bookings, reported by stay month.
- **Scale:** this version uses pandas and Parquet. Every transform works on whole columns, so it ports directly to BigQuery SQL. At scale the merge belongs in BigQuery, not Python memory.
- **With more time:** schema contracts, SCD2 history of booking versions, and a dbt project.

AI/RAG (optional):

- Use an LLM (Gemini via `ML.GENERATE_TEXT`) only to *suggest* mappings for new free-text values. A person approves them before they go into the config.
- Store embeddings of reviews and descriptions in BigQuery (`VECTOR_SEARCH`) keyed by `property_id`, so RAG answers join to governed revenue facts.
- Log model and prompt versions so results can be audited.